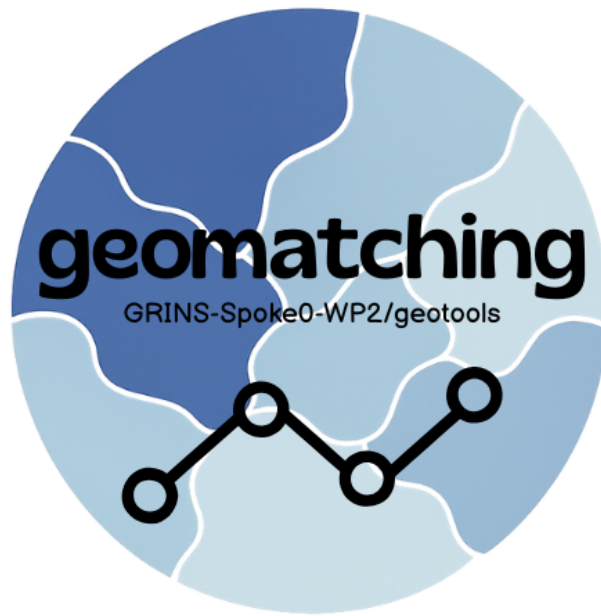




Tutorial to the usage of the AMELIA service:
geomatching

Alessandro Fusta Moro, Lorenzo Leoni, Alessandro Fassò

November 2025

Summary

The geomatching service available in AMELIA is designed to help users to harmonise spatio-temporal datasets onto a common spatial support. In particular, the spatial support corresponds to the administrative boundaries of Italy for 2025. This means that you can use any spatio-temporal grid or point-referenced data, and the geomatching service will project these variables onto the municipal scale. Datasets need to meet the requirements of the function. The service also provides intra-municipal (or provincial and regional) variability. After submitting a request, its status can be viewed on the status page under the Development Tools section in the AMELIA interface.

Introduction

Within the GRINS project, the AMELIA platform is where all the datasets have been collected and shared among the users. Many datasets are geo-referenced, meaning they are spatial or spatio-temporal data (e.g., temperature). However, other datasets are indexed by municipalities or regions, which come on different supports, namely administrative boundaries. To match these two types of data, a harmonisation technique is implemented in the geomatching service, which makes it possible to convert grids into municipal, provincial, or regional boundaries. This service allows researchers to match data spatially, enabling the use of statistical methods on the merged datasets. One possible outcome of the service is that, after the harmonisation of multiple datasets, the AMELIA platform will provide an enlarged municipal-level dataset collecting information from various domains such as environment, social aspects, and economics. This enriched dataset can guide policymakers towards a data-driven framework and enable deeper analyses of reality, uncovering connections between variables that can often meaningfully interact.

Input information

In order to use the service, your datasets need to be uploaded to AMELIA and must follow a precise format: they must contain information about space and time, for example two columns representing longitude and latitude. Once your datasets are on AMELIA, go to the **Services** section and access the **geomatching** service. You will need to select the datasets that have to be harmonised at the municipal scale, and specify which columns represent space (i.e., x and y) and which represent time. At the moment, the service does not support shapefiles that come with polygon-based spatial supports. After specifying x , y , and t , you need to indicate the columns that you want to harmonise. If no columns are specified, all columns will be harmonised by default. For each dataset you want to harmonise, you must specify three attributes: **format**, **type**, and **crs**. Finally, you need to choose the administrative level at which you want the final aggregation (i.e., municipal, provincial, or regional).

Format

The **format** explains how the data are organised. Two possible formats are available: **dataframe** and **matrix**. If the data come in a table with columns x , y , and t , where these three columns represent respectively the x-coordinate, the y-coordinate, and time, then your dataset is a dataframe and you need to select **xyt** in the corresponding field. If the dataset is a three-dimensional tensor, you should use the **matrix** option. Once you have defined the organisation of your data, you need to specify

whether your data represent point locations or a grid. In the latter case, the points indicate the centres of the grid cells and must be equally spaced. This selection is made in the **type** category.

Table 1: Example of dataframe "xyt"

Longitude	Latitude	time	temperature
6	41.75	2023-12-29	13.704
6	42	2023-12-29	14.032
6.1	41.75	2023-12-29	14.559
6.1	42	2023-12-29	13.338
6	41.75	2023-12-30	13.280
6	42	2023-12-30	13.512
6.1	41.75	2023-12-30	14.064
6.1	42	2023-12-30	14.789

Type

The **type** specifies what kind of spatial data you have. Every pair of coordinates can indicate either a point location or the centre of a grid cell, and we ask users to specify this because the results may vary. In particular, if the data represent point locations, the average at the municipal level will be computed as the mean of the values associated with the points belonging to that municipality (or province or region). In this case, you need to select **points**. If the data represent a grid, even if the dataset has the same structure, the averages for municipalities (or provinces or regions) are calculated by taking into account the fact that the input is a grid. Therefore, the resulting average will be different, as it considers the shape of each polygon (e.g., municipalities) and the area of the grid cells (also called pixels). In Table 1, the type of the data is **grid**.

crs

After you specify how your datasets are organised and what they represent, you need to indicate the coordinate reference system (**crs**) of your data, because the x - and y -coordinates may differ across coordinate reference systems for the same physical location. This field is automatically set to EPSG:4326. If you need to change it, simply specify the number after "EPSG:". In Table 1, the **crs** is the default (i.e., 4326).

Aggregation Level

In this section you need to specify the desired level of aggregation for the harmonisation. You can select among three different aggregation levels: **Municipal**, **Provincial**, or **Regional**.

Standard deviation (sd)

Specifies the type of uncertainty desired. Accepted values are **intra** or **varX**. Default is **intra** where the standard deviation is calculated as the intra-variability within the polygon. Use **varX** if the variable has a standard deviation associated. In this case, for each column harmonised, another column called the same plus "_sd" is looked for. For each polygon, these values are squared, summed

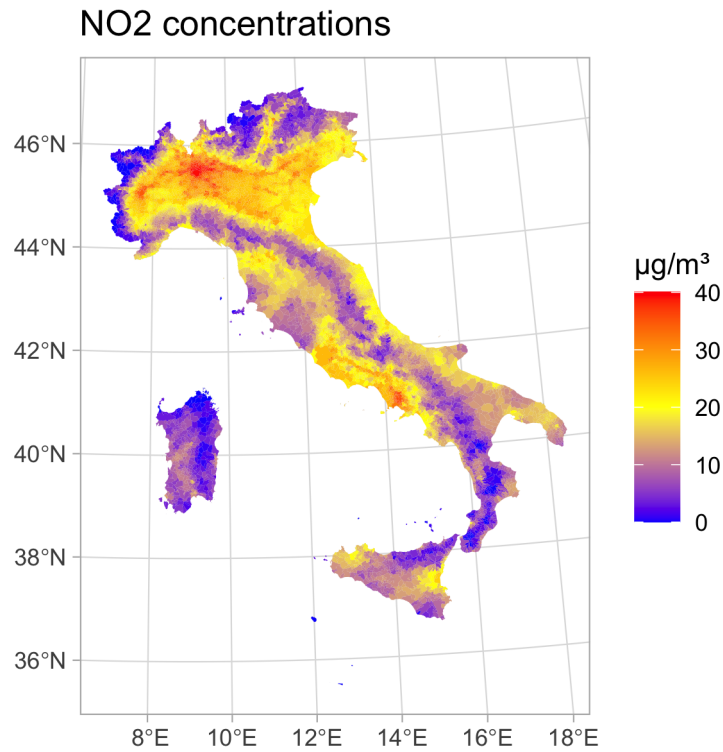


Figure 1: NO2 concentrations in Italy at 12th November 2023, at municipal level

and divided by the square of elements used. Finally, square root is provided. Use only with mean aggregation. Currently not calculated in the AMELIA service. Use the R package instead.

Statistics (stats)

Select which statistics you want to calculate. Use only if aggregate is **TRUE**. Accepted values are: **min**, **1st_quartile**, **mean**, **median**, **3rd_quartile**, **max**, **sd** or **ALL** if you want all. Default is **mean**. Currently not modifiable in the service. Use the R package instead.

Results

Status

After you have started the process, you can exit the service because the computation is performed in the background. You can monitor the status of your request on the **Status** page, in the *Development Tools* section of the AMELIA interface. On this page, different logs will be printed, including: whether the datasets are valid for the function, what the service is currently doing, whether there are any errors during the procedure, and the final output of the service. In case of an error, the message will indicate where the problem occurred (e.g., dataset not compliant", x-coordinate not

valid”, etc.). In case of success, a message confirming the completion of the operation will be printed. You will then find your harmonised datasets in the *My Processing* section.

Output datasets

The output dataset is saved in *My Processing* under the *Catalog* section. From there, the user can easily access the dataset, check its correctness, compute statistics, and eventually publish or delete it. Statistics can be calculated using the Jupyter Hub available in the AMELIA interface under the *Development Tools* section. The output dataset contains two reference columns identifying space and time. In particular, the space column is called `PRO_COM` when the aggregation level is municipal, `COD_PROV` for provincial aggregation, and `COD_REG` for regional aggregation. The time coordinate is always stored in the column `time`. Once you have checked the correctness of the dataset, you can publish it on AMELIA and download it. If you need to add geometries to your file, please download the administrative boundaries from ISTAT (https://www.istat.it/storage/cartografia/confini_amministrativi/generalizzati/2025/Limiti01012025_g.zip) and merge the dataset with the shapefile downloaded from ISTAT. Geometries can also be downloaded from the Zenodo page (<https://zenodo.org/records/17555473>). Below is an example of how the dataset can be extended with geometries in the case of municipalities.

Here is an example of how to retrieve geometries of municipalities in R

```
library(readr)
library(dplyr)
library(sf)

#importing LAUs geometries
download.file(
  "https://www.istat.it/storage/cartografia/confini_amministrativi/
  generalizzati/2025/Limiti01012025_g.zip",
  destfile = "IT_adm_lim.zip")
unzip("IT_adm_lim.zip")
metadata_GRINS_LAUs <- st_read("Com01012025_g", "Com01012025_g_WGS84")

#importing climate variables at municipal levels
GRINS_df_LAUs_Italy <- read.csv("yourdataframe.csv") #<- change to your
dataframe

#merging with geometries
GRINS_df_LAUs_Italy_geom <- left_join(metadata_GRINS_LAUs, subset_CL, by="PRO_
COM") #or COD_REG or COD_PROV
```

Working Example

In the following section, we provide a working example of the function **geomatching** also available as R package downloadable from GitLab:

```
library(devtools)
devtools::install_github("GRINS-Spoke0-WP2/geotools")
```

Import R libraries

```
rm(list = ls())
library(ggplot2)
library(mapdata)
library(geotools)
```

Import (local) data

- ERA5Land_t2m_10d: time-spatial data.frame (*longitude*, *latitude* and *time*) containing 10 days of **air temperature**. Grid resolution: $0.1^\circ \times 0.1^\circ$ (about 11 km x 11 km).
- ERA5SL_t2m_10d: time-spatial data.frame containing 10 days of **air temperature**. Grid resolution: $0.25^\circ \times 0.25^\circ$ (about 27 km x 27 km).
- ERA5SL_rh_10d: time-spatial data.frame containing 10 days of **relative humidity**. Grid resolution: $0.25^\circ \times 0.25^\circ$ (about 27 km x 27 km).
- ERA5SL_lai_hv_10d_matrix: 3D matrix (*x*-component represents *longitude*, *y*-component describes *latitude* and *z*-component represents *time*) 10 days of **Leaf Area Index - High Vegetation**. Grid resolution: $0.25^\circ \times 0.25^\circ$ (about 27 km x 27 km).

```
# weather data
load("WE_raw_data.Rdata")
names(ERA5Land_t2m_10d)[4] <- "t2m_Land"
names(ERA5SL_t2m_10d)[4] <- "t2m_SL"
WE_data = list(
  "ERA5Land_t2m" = list(
    "data" = ERA5Land_t2m_10d,
    "var" = "t2m_Land",
    "label" = "Air temperature, ERA5Land (11 km x 11 km)"
  ),
  "ERA5SL_t2m" = list(
    "data" = ERA5SL_t2m_10d,
    "var" = "t2m_SL",
    "label" = "Air temperature, ERA5SL (27 km x 27 km)"
  ),
  "ERA5SL_rh" = list(
    "data" = ERA5SL_rh_10d,
    "var" = "rh",
    "label" = "Relative humidity, ERA5SL (27 km x 27 km)"
  )
)
```

```

# LAUs boundaries
names(mun_bounds)[6] <- "COD_COM"
LAUs_data = list(
  "mun_bounds" = list(
    "data" = geotools::mun_bounds,
    "code" = "mun",
    "join_var" = "PRO_COM"
  ),
  "prov_bounds" = list(
    "data" = geotools::prov_bounds,
    "code" = "prov",
    "join_var" = "COD_PROV"
  ),
  "reg_bounds" = list(
    "data" = geotools::reg_bounds,
    "code" = "reg",
    "join_var" = "COD_REG"
  )
)

```

Visualize first rows of each data.frame

ERA5Land_t2m_10d

```
head(ERA5Land_t2m_10d, n = 10)
```

```

##      longitude latitude      time t2m_Land
## 1           6      48.0 2013-01-01 5.308351
## 2           6      47.9 2013-01-01 5.617759
## 3           6      47.8 2013-01-01 5.829429
## 4           6      47.7 2013-01-01 6.035239
## 5           6      47.6 2013-01-01 6.123781
## 6           6      47.5 2013-01-01 6.104250
## 7           6      47.4 2013-01-01 6.273520
## 8           6      47.3 2013-01-01 6.243166
## 9           6      47.2 2013-01-01 5.943280
## 10          6      47.1 2013-01-01 5.592205

```

ERA5SL_t2m_10d

```
head(ERA5SL_t2m_10d, n = 10)
```

```

##      longitude latitude      time  t2m_SL
## 1           6      48.00 2013-01-01 5.631985
## 2           6      47.75 2013-01-01 6.295315
## 3           6      47.50 2013-01-01 6.754544
## 4           6      47.25 2013-01-01 6.592516
## 5           6      47.00 2013-01-01 5.405178
## 6           6      46.75 2013-01-01 3.390530
## 7           6      46.50 2013-01-01 2.423489
## 8           6      46.25 2013-01-01 2.588853

```

```
## 9          6      46.00 2013-01-01 2.084459
## 10         6      45.75 2013-01-01 1.991034
```

ERA5SL_rh_10d

```
head(ERA5SL_rh_10d, n = 10)
```

```
##   longitude latitude      time      rh
## 1         6.00      48 2013-01-01 88.84715
## 2         6.25      48 2013-01-01 87.67344
## 3         6.50      48 2013-01-01 86.28634
## 4         6.75      48 2013-01-01 85.53499
## 5         7.00      48 2013-01-01 86.53418
## 6         7.25      48 2013-01-01 90.22042
## 7         7.50      48 2013-01-01 92.99447
## 8         7.75      48 2013-01-01 93.77895
## 9         8.00      48 2013-01-01 89.41314
## 10        8.25      48 2013-01-01 83.32842
```

First look

```
for (item_i in WE_data) {

  # configuration
  df_i <- item_i$data
  var_i <- item_i$var
  label_i <- item_i$label
  subset_i <- df_i[df_i$time == df_i$time[1], ]

  # run
  p <- ggplot() +
    geom_map(
      data = map_data("italy"),
      map = map_data("italy"),
      aes(map_id = region),
      fill = "lightgrey",
      color = "black",
      linewidth = .1,
      alpha=.9
    ) +
    geom_tile(
      data = subset_i,
      aes(x = longitude, y = latitude, fill = .data[[var_i]])
    ) +
    labs(
      title = sprintf("Geographical map of weather data"),
      subtitle = label_i,
      x = "Longitude",
      y = "Latitude"
    ) +
    scale_fill_viridis_c(
```

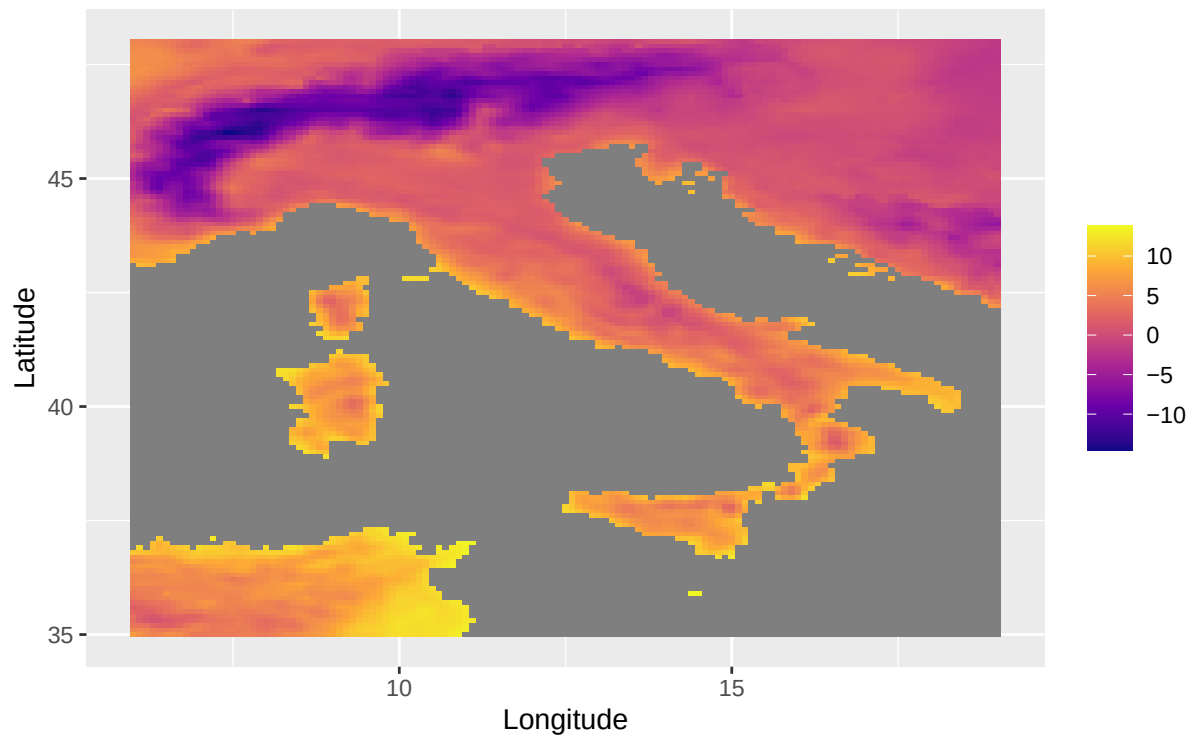
```

    option = "C"
  ) +
  theme(
    legend.title = element_blank(),
    plot.title = element_text(face = "bold")
  )
print(p)
}

```

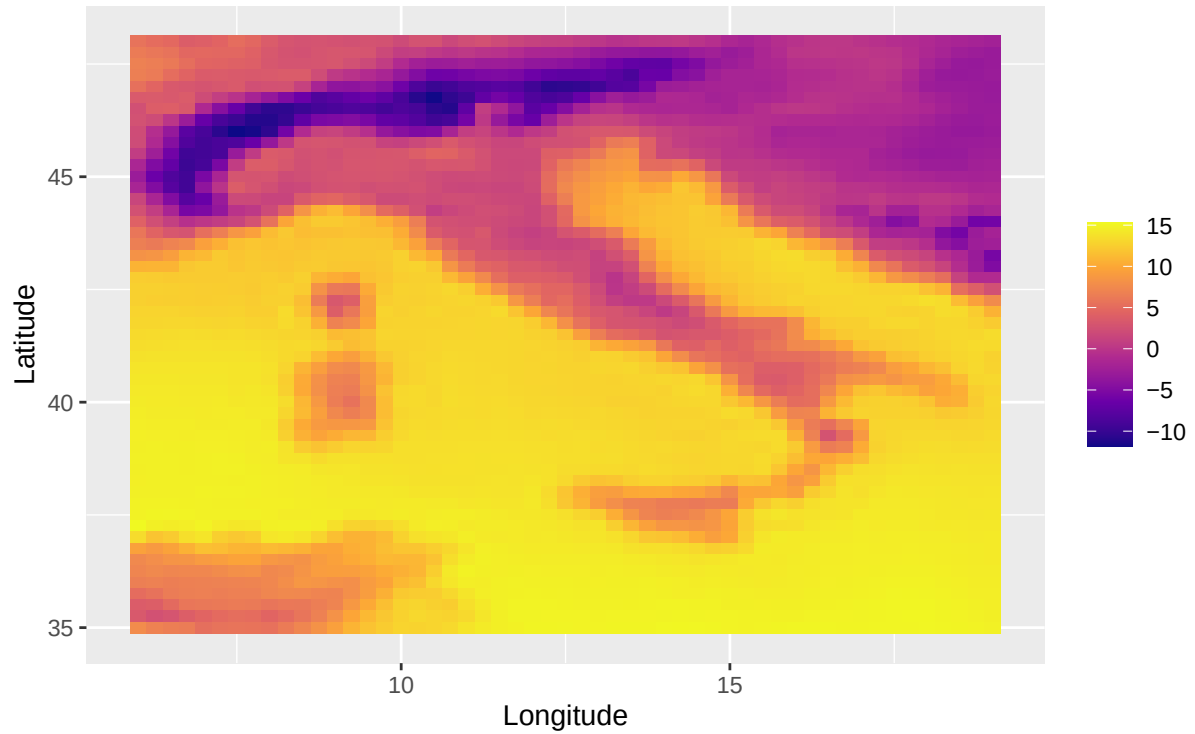
Geographical map of weather data

Air temperature, ERA5Land (11 km x 11 km)



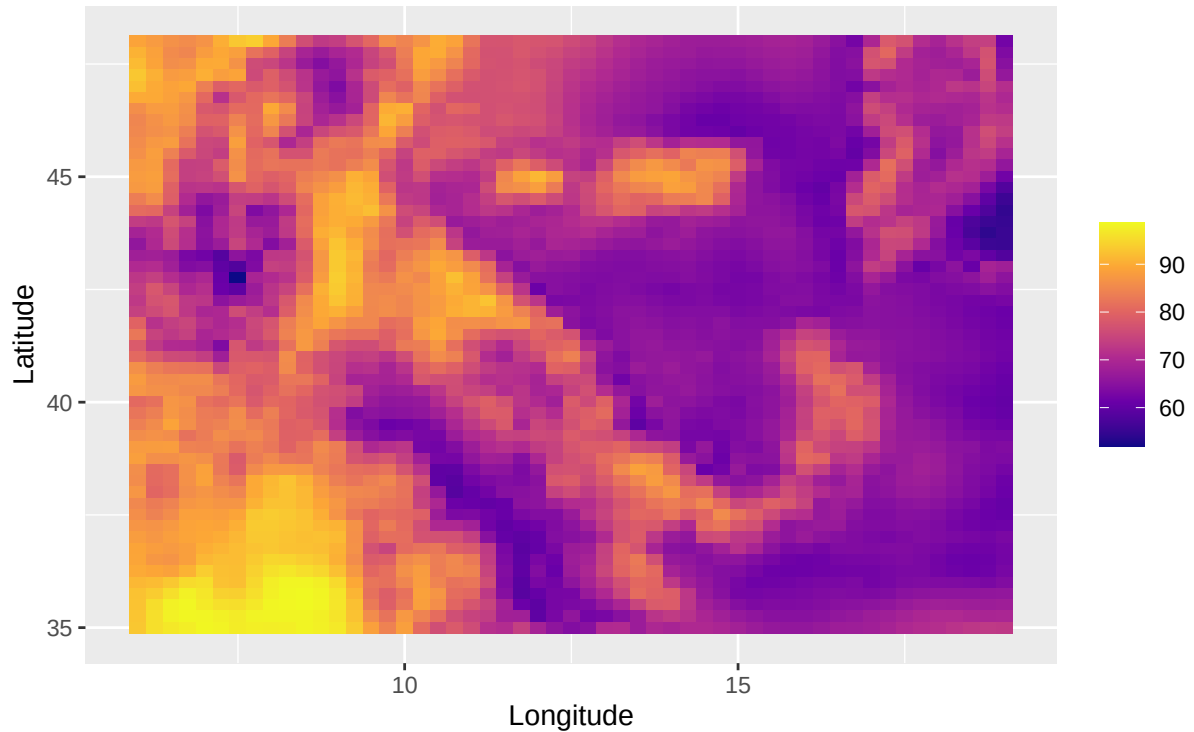
Geographical map of weather data

Air temperature, ERA5SL (27 km x 27 km)



Geographical map of weather data

Relative humidity, ERA5SL (27 km x 27 km)



Run geomatching

```
aggr_data <- list()
for (item_i in LAUs_data) {

  # configuration
  bounds_i <- item_i$data
  code_i <- item_i$code
  join_var_i <- item_i$join_var

  # run
  temp <- geotools::geomatching(
    data = list(
      ERA5Land_t2m_10d,
      ERA5SL_t2m_10d,
      ERA5SL_rh_10d,
      ERA5SL_lai_hv_10d_matrix
    ),
    settings = list(
      "format" = list(
        "xyt",
        "xyt",
        "xyt",
        "matrix"
      )
    )
  )
}
```

```

    ),
    "type" = list(
      "grid",
      "grid",
      "grid",
      "grid"
    ),
    "crs" = list(
      4326,
      4326,
      4326,
      4326
    )
  ),
  aggregate = TRUE,
  group_by = code_i,
  stats = c(
    "min",
    "1st_quartile",
    "mean",
    "median",
    "3rd_quartile",
    "max",
    "sd"
  )
)
)

# rename columns #1
names(temp)[24:30] <- c(
  "lai_hv_min",
  "lai_hv_1st_quartile",
  "lai_hv_mean",
  "lai_hv_median",
  "lai_hv_3rd_quartile",
  "lai_hv_max",
  "lai_hv_std"
)

# fill missing values
stats <- c("min", "1st_quartile", "mean", "median", "3rd_quartile", "max", "sd")
for (stat_i in stats) {
  land_col <- paste0("t2m_Land_", stat_i)
  sl_col <- paste0("t2m_SL_", stat_i)
  temp[[land_col]][is.na(temp[[land_col]])] <- temp[[sl_col]][is.na(temp[[land_col]])]
}

# drop columns
temp <- temp[, !names(temp) %in% c(
  "t2m_SL_min", "t2m_SL_1st_quartile", "t2m_SL_mean", "t2m_SL_median",
  "t2m_SL_3rd_quartile", "t2m_SL_max", "t2m_SL_sd"
)]

# rename columns #2

```



```

names(temp)[3:9] <- c(
  "t2m_min",
  "t2m_1st_quartile",
  "t2m_mean",
  "t2m_median",
  "t2m_3rd_quartile",
  "t2m_max",
  "t2m_sd"
)

# join
temp <- left_join(
  bounds_i,
  temp,
  by = join_var_i
)

# save
aggr_data[[code_i]] <- temp
}

```

Visualize aggregated air temperature

```

for (code_i in names(aggr_data)) {

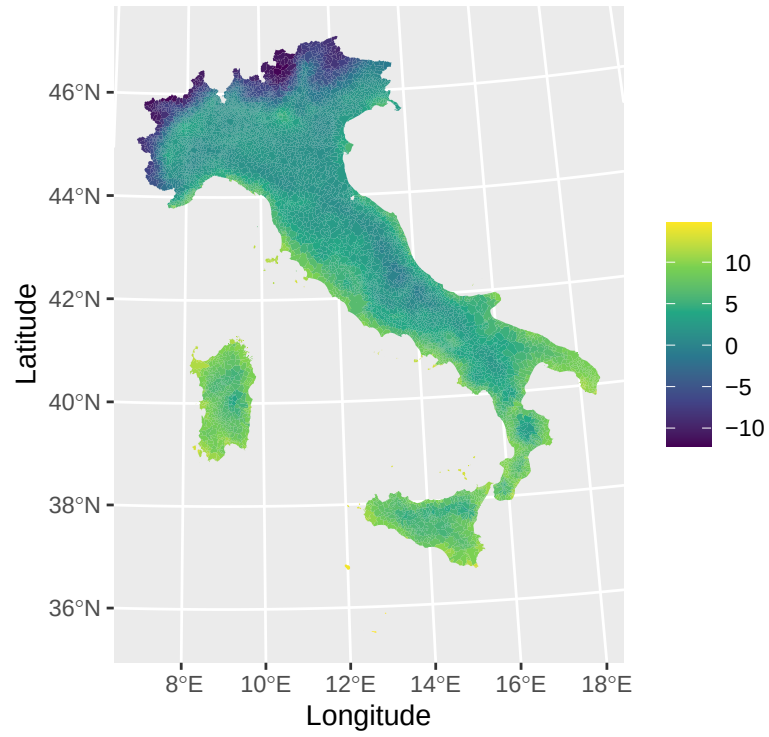
  # configuration
  df_i <- aggr_data[[code_i]]
  subset_i <- df_i[df_i$time == df_i$time[1], ]

  # run
  p <- ggplot() +
    geom_sf(
      data = subset_i,
      aes(fill = t2m_mean),
      color = NA) +
    scale_fill_continuous(type = "viridis", name = "Temperature [°C]") +
    labs(
      title = "Geographical map of air temperature",
      subtitle = sprintf("Aggregation level: %s", code_i),
      x = "Longitude",
      y = "Latitude") +
    theme(
      legend.title = element_blank(),
      plot.title = element_text(face = "bold")
    )
  print(p)
}

```

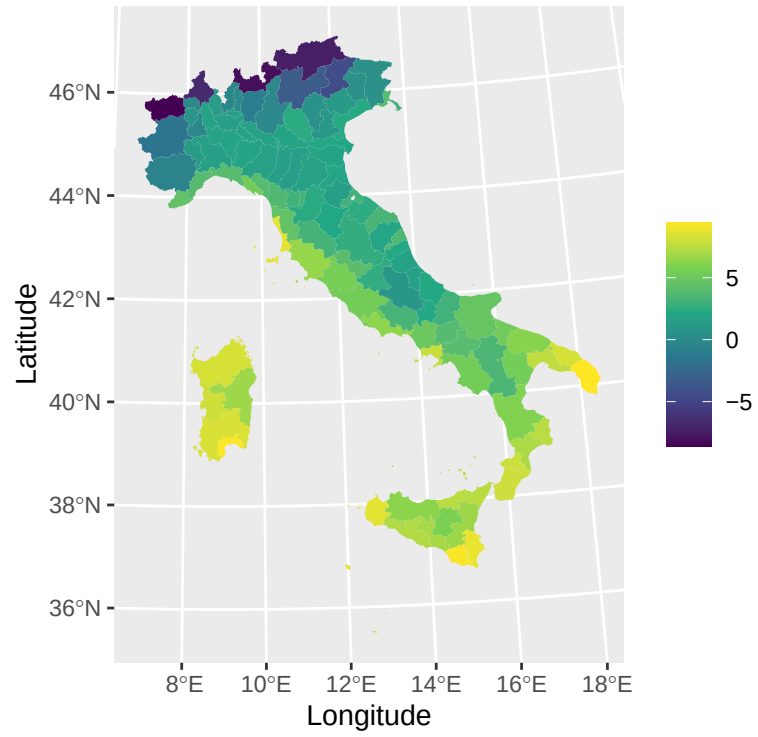
Geographical map of air temperature

Aggregation level: mun



Geographical map of air temperature

Aggregation level: prov



Geographical map of air temperature

Aggregation level: reg

